

Fiche 305 — Tableaux et matrices : `dim`, produit extérieur, algèbre linéaire

Matière	Code · Langage R
Cours source	R Core Team, <i>An Introduction to R</i> 4.6.1 — chapitre 5 « Arrays and matrices » (§5.1 tableaux, §5.2 indexation, §5.3 matrices d'index, §5.4 <code>array()</code> et recyclage, §5.5 produit extérieur, §5.6 transposition généralisée, §5.7 facilités matricielles, §5.8 <code>cbind</code> / <code>rbind</code> , §5.9 <code>c()</code> , §5.10 tables de fréquences)
Sources d'appoint	<i>R Language Definition</i> 4.6.1, §2.2.2 « Dimensions », §2.2.3 « Dimnames », §3.4.2 « Indexing matrices and arrays »
Difficulté	Avancé — long chapitre, dense en fonctions et en pièges numériques
Temps d'étude estimé	2 h
Prérequis	Fiches 301 à 304 (recyclage, indexation, attributs, facteurs)
Concepts clés	vecteur de dimension, ordre colonne-majeur , <code>dim()</code> , indice vide, indice unique, matrice d'index , <code>array()</code> , les cinq règles du recyclage mixte, <code>%%</code> et <code>outer()</code> , <code>aperm()</code> et <code>t()</code> , <code>nrow</code> / <code>ncol</code> , <code>%*%</code> , <code>crossprod()</code> , les trois sens de <code>diag()</code> , <code>solve()</code> , <code>eigen()</code> , <code>svd()</code> , <code>det()</code> , <code>qr()</code> , <code>cbind()</code> / <code>rbind()</code> , <code>as.vector()</code> , <code>table()</code> et <code>cut()</code>
À retenir en priorité	Colonne-majeur · <code>solve(A,b)</code> , jamais <code>solve(A) %*% b</code> · les trois sens de <code>diag()</code> · <code>c()</code> efface <code>dim</code> , <code>cbind</code> / <code>rbind</code> le respectent · la matrice d'index .

Vue d'ensemble

UN TABLEAU	un vecteur + un VECTEUR DE DIMENSION (l'attribut <code>dim</code>) longueur du vecteur de <code>dim = k</code> -> tableau a k dimensions une MATRICE est un tableau a 2 dimensions
ORDRE	« COLUMN MAJOR ORDER » -- comme en FORTRAN le PREMIER indice varie le plus vite, le DERNIER le plus lentement <code>dim = c(3,4,2)</code> -> <code>a[1,1,1]</code> , <code>a[2,1,1]</code> , ... , <code>a[2,4,2]</code> , <code>a[3,4,2]</code>
INDEXER	<code>a[2,,]</code> indice vide = TOUTE l'étendue

a[,]
a[i]

le tableau entier, comme a tout seul
un SEUL indice -> dim IGNORE, sauf si i est une MATRICE

MATRICE D'INDEX autant de colonnes que de dimensions, autant de lignes qu'on veut
x[i] EXTRAIT une collection irreguliere
x[i] <- 0 la REMPLACE
negatifs INTERDITS ; ligne a zero IGNOREE ; ligne a NA -> NA

CONSTRUIRE array(donnees, dim) RECYCLE si trop court
dim(h) <- c(3,4,2) ERREUR si la longueur ne colle pas

PRODUIT EXTERIEUR a %o% b = outer(a, b, "%*") -- fonction remplaçable

ALGEBRE t() aperm() nrow() ncol() %%% crossprod()
solve(A,b) POUR RESOUDRE solve(A) pour inverser (rarement utile)
eigen() svd() det() determinant() qr() lsfit()

ASSEMBLER cbind() rbind() RESPECTENT dim | c() l'EFFACE

COMPTER table(f) table(f1, f2) -- k facteurs -> tableau k-dimensionnel

Le problème posé. « *An array can be considered as **a multiply subscripted collection of data entries*** » (§5.1). Le chapitre 4 s'achevait sur une promesse : quand les sous-classes ont **toutes la même taille**, « *the indexing may be done **implicitly and much more efficiently*** ». Le tableau est cette structure — et le prix à payer est **la régularité**.

INTUITION — LE TABLEAU N'AJOUTE RIEN AUX DONNÉES.

« **A vector can be used by R as an array only if it has a dimension vector as its `dim` attribute.** » Un tableau, c'est un vecteur **plus une façon de compter**. `dim(z) <- c(3,5,100)` sur un vecteur de 1500 éléments « *gives it the `dim` attribute that allows it to be treated as a 3 by 5 by 100 array* » — sans déplacer un seul nombre.

● Concept 1 — Le vecteur de dimension et l'ordre colonne-majeur

DÉFINITION (§5.1).

« **A dimension vector is a vector of non-negative integers. If its length is k then the array is k -dimensional** — e.g. a matrix is a 2-dimensional array. **The dimensions are indexed from one up to the values given in the dimension vector.** »

```
z                      # un vecteur de 1500 elements
dim(z) <- c(3, 5, 100) # z devient un tableau 3 x 5 x 100
```

« Other functions such as `matrix()` and `array()` are available for **simpler and more natural looking assignments** » (§5.4).

Règle — l'ordre de remplissage (§5.1). « *The values in the data vector give the values in the array in the same order as they would occur in FORTRAN, that is "column major order," with the first subscript moving fastest and the last subscript slowest.* »

« For example if the dimension vector for an array `a` is `c(3,4,2)` then there are $3 \times 4 \times 2 = 24$ entries in `a` and the data vector holds them in the order

`a[1,1,1], a[2,1,1], ..., a[2,4,2], a[3,4,2]`

»

Comment retenir l'ordre. Le **premier** indice tourne le plus vite : on descend la colonne avant de passer à la suivante. Sur une matrice 3×4 remplie de `1:12` :

$$\begin{pmatrix} 1 & 4 & 7 & 10 \\ 2 & 5 & 8 & 11 \\ 3 & 6 & 9 & 12 \end{pmatrix}$$

C'est l'inverse de la lecture naturelle d'un tableau imprimé. Un débutant qui fabrique une matrice « ligne par ligne » avec `matrix(v, nrow = 3)` obtient sa transposée. L'argument à connaître est `byrow = TRUE` — déjà utilisé par le manuel en fiche 302.

COMPLÉMENT (R LANGUAGE DEFINITION §2.2.2).

« *R ensures that the length of the vector is the product of the lengths of the dimensions. The length of one or more dimensions may be zero.* »

« Arrays can be **one-dimensional** : such arrays are **usually treated in the same way as vectors** (including when printing), **but the exceptions can cause confusion.** » (§5.1) — les exceptions vues en fiche 302 : le `dim` de longueur 1, et `names` qui accède en fait à `dimnames[[1]]` .

● Concept 2 — Indexer un tableau

DÉFINITION (§5.2).

« Individual elements of an array may be referenced by giving **the name of the array followed by the subscripts in square brackets, separated by commas.** »

« More generally, **subsections of an array may be specified by giving a sequence of index vectors in place of subscripts** ; however **if any index position is given an empty index vector, then the full range of that subscript is taken.** »

L'exemple du cours, sur le tableau `a` de dimension `c(3,4,2)` : « `a[2,,]` is a **4 by 2 array** with dimension vector `c(4,2)` and data vector containing the values

```
c(a[2,1,1], a[2,2,1], a[2,3,1], a[2,4,1],  
  a[2,1,2], a[2,2,2], a[2,3,2], a[2,4,2])
```

in that order. »

CONTRÔLE — RETROUVER CET ORDRE SOI-MÊME.

Le résultat est un tableau 4×2 : par la règle colonne-majeur, son **premier** indice (celui de longueur 4, hérité de la 2^e dimension de `a`) tourne le plus vite. On parcourt donc `a[2,1,1]` , `a[2,2,1]` , `a[2,3,1]` , `a[2,4,1]` — **la 2^e dimension d'abord** — avant de passer à la 3^e. conforme.

Et **la dimension de longueur 1 a disparu** : `a[2,,]` n'est pas $1 \times 4 \times 2$ mais 4×2 . C'est la règle `drop` de la fiche 302, ici à l'œuvre sans qu'on l'ait demandée.

« `a[, ]` stands for the entire array, which is the same as **omitting the subscripts entirely and using `a` alone**. »

« For any array `Z`, the dimension vector may be referenced explicitly as `dim(Z)` (on either side of an assignment). »

La règle de l'indice unique (§5.2), et son exception. « if an array name is given with **just one subscript or index vector**, then **the corresponding values of the data vector only are used** ; in this case the dimension vector is ignored. This is not the case, however, if the single index is not a vector but itself an array, as we next discuss. »

`x[i]` avec `i` **vecteur** → on lit le vecteur sous-jacent, `dim` mis de côté. `x[i]` avec `i` **matrice** → tout autre chose : concept 3.

● Concept 3 — La matrice d'index

DÉFINITION (§5.3).

« As well as an index vector in any subscript position, **a matrix may be used with a single index matrix** in order **either to assign a vector of quantities to an irregular collection of elements in the array, or to extract an irregular collection as a vector**. »

« In the case of a doubly indexed array, an index matrix may be given consisting of **two columns and as many rows as desired**. The entries in the index matrix are the row and column indices for the doubly indexed array. »

L'exemple complet du cours. « Suppose we have a 4 by 5 array `x` and we wish to : **extract elements `x[1,3]` , `x[2,2]` and `x[3,1]` as a vector structure**, and **replace these entries in the array `x` by zeroes**. In this case we need a **3 by 2 subscript array**. »

```
x <- array(1:20, dim = c(4,5))  # un tableau 4 x 5
x
#      [,1] [,2] [,3] [,4] [,5]
# [1,]    1    5    9   13   17
# [2,]    2    6   10   14   18
# [3,]    3    7   11   15   19
```

```
# [4,]    4    8   12   16   20

i <- array(c(1:3, 3:1), dim = c(3,2)) # i est un tableau d'index 3 x 2
i
#      [,1] [,2]
# [1,]    1    3
# [2,]    2    2
# [3,]    3    1

x[i]      # extraire ces elements
# [1] 9 6 3

x[i] <- 0  # les remplacer par des zeros
x
#      [,1] [,2] [,3] [,4] [,5]
# [1,]    1    5    0   13   17
# [2,]    2    0   10   14   18
# [3,]    0    7   11   15   19
# [4,]    4    8   12   16   20
```

CONTRÔLE — VÉRIFIER LES TROIS VALEURS EXTRAITES.

La matrice est remplie **en colonne-majeur** par `1:20`, donc la colonne j commence à $4(j-1) + 1$ et l'élément (l, j) vaut $4(j-1) + l$.

Ligne de i	Coordonnées	Valeur $4(j-1) + l$
1	(1,3)	$4 \times 2 + 1 = 9$
2	(2,2)	$4 \times 1 + 2 = 6$
3	(3,1)	$4 \times 0 + 3 = 3$

`x[i]` vaut bien `9 6 3`. Et l'affichage après remplacement confirme que ce sont exactement ces trois cases qui sont passées à 0.

Les restrictions (§5.3), identiques à celles du §3.4.2. « *Negative indices are not allowed in index matrices.* NA and zero values *are* allowed : rows in the index matrix containing a zero are ignored, and rows containing an NA produce an NA in the result. »

Et une restriction supplémentaire, propre au §5.3 : « *Index matrices must be numerical : any other form of matrix (e.g. a logical or character matrix) supplied as a matrix is treated as an indexing vector.* » Une matrice logique passée à `x[...]` **n'est pas** une matrice d'index — elle retombe dans le cas « indice unique » du concept 2.

Énoncé (§5.3). « *As a less trivial example, suppose we wish to generate an (unreduced) design matrix for a block design defined by factors blocks (b levels) and varieties (v levels). Further suppose there are n plots in the experiment.* »

```
Xb <- matrix(0, n, b)
Xv <- matrix(0, n, v)
ib <- cbind(1:n, blocks)
iv <- cbind(1:n, varieties)
Xb[ib] <- 1
Xv[iv] <- 1
X <- cbind(Xb, Xv)
```

Étape 1 — ce qu'on veut. Une matrice **d'indicatrices** : une ligne par parcelle, une colonne par bloc, avec un **1** dans la colonne du bloc auquel la parcelle appartient.

Étape 2 — préparer la toile. `matrix(0, n, b)` crée une matrice $n \times b$ **entièrement nulle** — l'usage extrême dont parle le §5.4 (« *an extreme but common example* »).

Étape 3 — fabriquer les coordonnées. `cbind(1:n, blocks)` empile deux colonnes : **le numéro de ligne et le numéro de bloc**. C'est exactement la forme requise — « *two columns and as many rows as desired* », ici n lignes.

Étape 4 — allumer les cases. `Xb[ib] <- 1` pose un 1 aux n coordonnées d'un coup. **Aucune boucle.**

Étape 5 — assembler. `cbind(Xb, Xv)` accole les deux blocs d'indicatrices horizontalement (concept 8).

Étape 6 — la matrice d'incidence. « To construct the incidence matrix, `N` say, we could use `N <- crossprod(Xb, Xv)` » — soit $X_b^T X_v$, dont l'élément (i, j) compte les parcelles à la fois dans le bloc i et de la variété j .

Étape 7 — et la voie courte. « **However a simpler direct way of producing this matrix is to use `table()`** : `N <- table(blocks, varieties)` » (concept 10). **Le même résultat en une fonction**, sans construire les indicatrices.

Étape 8 — la leçon. L'exemple long enseigne **le mécanisme** ; la ligne courte est ce qu'on écrit en pratique. Le cours donne les deux **expres** : `crossprod` d'indicatrices **est** un comptage croisé, et le comprendre éclaire ce que fait un modèle linéaire sur des facteurs (fiche 313).

Remarque de lecture : `blocks` et `varieties` sont ici utilisés directement comme indices de colonne. C'est le point où le piège de la fiche 304 guette — si ce sont des facteurs, ce sont **leurs codes entiers** qui servent d'indices. Cela fonctionne parce que les codes sont précisément $1, \dots, b$ et $1, \dots, v$; mais la R Language Definition rappelle que s'appuyer là-dessus est un pari sur l'implémentation.

● Concept 4 — `array()` et la règle du recyclage mixte

DÉFINITION (§5.4).

« As well as giving a vector structure a `dim` attribute, arrays can be constructed from vectors by the `array` function, which has the form `Z <- array(data_vector, dim_vector)` . »

La différence décisive entre les deux voies :

Écriture	Si le vecteur est plus court que le produit des dimensions
<code>Z <- array(h, dim = c(3,4,2))</code>	« its values are recycled from the beginning again to make it up to size 24 »
<code>dim(h) <- c(3,4,2)</code>	« would signal an error about mismatching length »

« If the size of `h` is **exactly 24** the result is the same as `Z <- h ; dim(Z) <- c(3,4,2)` . » — les deux voies coïncident **seulement** dans ce cas.

« As an **extreme but common example**, `Z <- array(0, c(3,4,2))` makes `Z` an array of all zeros. » C'est le cas limite du recyclage : **un** zéro recyclé 24 fois.

« At this point `dim(Z)` stands for the dimension vector `c(3,4,2)`, `Z[1:24]` stands for the data vector as it was in `h`, and `Z[]` with an empty subscript or `Z` with no subscript stands for the entire array as an array. »

4.1 Arithmétique sur les tableaux

Règle (§5.4). « Arrays may be used in arithmetic expressions and the result is an array formed by **element-by-element operations on the data vector**. The `dim` attributes of operands generally need to be the same, and this becomes the dimension vector of the result. »

```
D <- 2*A*B + C + 1      # si A, B, C sont des tableaux semblables
```

4.2 Les cinq règles du mélange vecteur / tableau

Le cours prévient (§5.4.1). « The precise rule affecting element by element mixed calculations with vectors and arrays is **somewhat quirky and hard to find in the references**. From experience we have found the following to be a reliable guide. »

#	Règle (§5.4.1)
1	« The expression is scanned from left to right . »
2	« Any short vector operands are extended by recycling their values until they match the size of any other operands. »
3	« As long as short vectors and arrays only are encountered, the arrays must all have the same <code>dim</code> attribute or an error results . »
4	« Any vector operand longer than a matrix or array operand generates an error . »
5	« If array structures are present and no error or coercion to vector has been precipitated, the result is an array structure with the common <code>dim</code> attribute of its array operands . »

EN CLAIR.

Les règles 2 et 4 sont **asymétriques** : un vecteur **plus court** qu'un tableau est **recyclé en silence** ; un vecteur **plus long** est **une erreur**. Le premier cas est celui qui blesse — un vecteur de longueur 3 ajouté à une matrice 4×5 est recyclé sur 20 cases, **en colonne-majeur**, et le résultat n'a aucun sens tout en ayant la bonne forme.

La règle 1 (« *scanned from left to right* ») explique pourquoi l'ordre des opérandes peut changer le comportement : c'est le **premier** tableau rencontré qui fixe la forme attendue.

● Concept 5 — Le produit extérieur

DÉFINITION (§5.5).

*« If `a` and `b` are two numeric arrays, **their outer product is an array whose dimension vector is obtained by concatenating their two dimension vectors (order is important)**, and whose data vector is got by **forming all possible products of elements of the data vector of `a` with those of `b`**. »*

```
ab <- a %o% b
ab <- outer(a, b, "*")    # equivalent
```

Règle — la généralisation (§5.5). « *The multiplication function can be replaced by an arbitrary function of two variables.* »

L'exemple du cours — celui de l'annexe A (fiche 300), ici expliqué :

```
f <- function(x, y) cos(y)/(1 + x^2)
z <- outer(x, y, f)
```

« if we wished to **evaluate the function $f(x; y) = \cos(y)/(1 + x^2)$ over a regular grid of values** with *x*- and *y*-coordinates defined by the R vectors `x` and `y` ».

C'est le remplacement idiomatique de la double boucle. `outer` évalue `f` sur **toutes les paires** (x_i, y_j) et range le résultat dans une matrice indexée par *i* et *j*.

« In particular **the outer product of two ordinary vectors is a doubly subscripted array** (that is a matrix, of rank at most 1). Notice that the outer product operator is of course non-commutative. »

Énoncé (§5.5). « As an **artificial but cute example**, consider the determinants of 2 by 2 matrices $\begin{pmatrix} a & b \\ c & d \end{pmatrix}$ where **each entry is a non-negative integer in the range 0, 1, ..., 9, that is a digit**. The problem is to find the determinants $ad - bc$ of **all possible matrices of this form** and represent **the frequency with which each value occurs** as a high density plot. This amounts to finding **the probability distribution of the determinant if each digit is chosen independently and uniformly at random**. »

« A neat way of doing this uses the `outer()` function twice » :

```
d <- outer(0:9, 0:9)
fr <- table(outer(d, d, "-"))
plot(fr, xlab = "Determinant", ylab = "Frequency")
```

Étape 1 — le premier `outer`. `outer(0:9, 0:9)` sans troisième argument utilise **la multiplication** par défaut : `d` est la matrice 10×10 de **tous les produits $a \times d$** , soit **100 valeurs**.

Étape 2 — pourquoi le même objet sert pour ad et pour bc . Les deux produits parcourent **exactement le même ensemble** de 100 valeurs, avec les mêmes multiplicités. Un seul `d` suffit donc pour les deux.

Étape 3 — le second `outer`. `outer(d, d, "-")` forme **toutes les différences** entre un élément de `d` et un élément de `d` : $100 \times 100 = 10\,000$ déterminants — soit exactement le nombre de quadruplets (a, b, c, d) possibles, 10^4 .

Étape 4 — compter. `table()` (concept 10) dénombre chaque valeur distincte du déterminant.

Étape 5 — tracer, et l'observation du cours sur `plot`. « Notice that `plot()` **here uses a histogram like plot method, because it "sees" that `fr` is of class "table"** » — le dispatch S3 de la fiche 311, exactement comme le `plot(Ecpt, Speed)` de l'annexe A.

Étape 6 — la remarque de performance. « **The "obvious" way of doing this problem with `for` loops ... is so inefficient as to be impractical**. » Quatre boucles imbriquées sur 10 valeurs, c'est 10^4 itérations **interprétées** ; deux `outer`, c'est 10^4 opérations **vectorisées** (fiche 301).

Étape 7 — le résultat surprenant. « **It is also perhaps surprising that about 1 in 20 such matrices is singular**. »

ENRICHISSEMENT PÉDAGOGIQUE (HORS COURS) — VÉRIFIER CE « 1 SUR 20 ».

Le cours donne le chiffre sans le justifier ; il se calcule exactement. Une matrice est singulière quand $ad = bc$. Si $N(p)$ désigne le nombre de couples $(a, d) \in \{0, \dots, 9\}^2$ de produit p , le nombre de quadruplets singuliers vaut $\sum_p N(p)^2$.

Le terme dominant est $p = 0$: $N(0) = 19$ (dix couples avec $a = 0$, dix avec $d = 0$, moins le couple $(0, 0)$ compté deux fois), soit **361** quadruplets à lui seul. En sommant sur tous les produits, on obtient **570** quadruplets singuliers sur 10 000, soit **5,7 %** — bien « *about 1 in 20* ».

Et l'essentiel est là : près des deux tiers des cas singuliers viennent du **zéro**. Le résultat ne dit pas que les matrices à chiffres sont souvent dégénérées par accident, il dit que **le zéro est une valeur très spéciale**. *Ce calcul n'est pas dans le cours ; il en confirme le chiffre.*

● Concept 6 — Transposer, et transposer en dimension quelconque

DÉFINITION (§5.6).

*« The function `aperm(a, perm)` may be used to **permute an array** `a`. The argument `perm` **must be a permutation of the integers** $\{1, \dots, k\}$, where k is the number of subscripts in `a`. The result of the function is **an array of the same size as** `a` **but with old dimension given by** `perm[j]` **becoming the new j -th dimension.** »*

*« **The easiest way to think of this operation is as a generalization of transposition for matrices.** Indeed if `A` is a matrix, then `B <- aperm(A, c(2,1))` **is just the transpose of** `A`. For this special case **a simpler function** `t()` **is available.** »*

Besoin	Fonction
Transposer une matrice	<code>t(A)</code>
Permuter les dimensions d'un tableau quelconque	<code>aperm(a, perm)</code>

● Concept 7 — Les facilités matricielles

Cadragé (§5.7). « *As noted above, a matrix is just an array with two subscripts. However it is such an important special case it needs a separate discussion. R contains many operators and functions that are available only for matrices.* »

Les trois de base : `t(X)` la transposée, `nrow(A)` et `ncol(A)` « *give the number of rows and columns in the matrix A* ».

7.1 Multiplier

NOTATION (§5.7.1).

« *The operator `%%` is used for matrix multiplication.* »

Écriture	Résultat
<code>A * B</code>	« <i>the matrix of element by element products</i> »
<code>A %% B</code>	« <i>the matrix product</i> »
<code>x %% A %% x</code>	« <i>a quadratic form</i> »

La promotion automatique des vecteurs (§5.7.1). « *An n by 1 or 1 by n matrix may of course be used as an n -vector if in the context such is appropriate. Conversely, **vectors which occur in matrix multiplication expressions are automatically promoted either to row or column vectors, whichever is multiplicatively coherent, if possible** (although **this is not always unambiguously possible**).* »

L'ambiguïté, en note 1 du §5.7.1 — le passage le plus subtil du chapitre. « *Note that `x %*% x` is ambiguous, as it could mean either $x^T x$ or xx^T , where x is the column form. In such cases the smaller matrix seems implicitly to be the interpretation adopted, so the scalar $x^T x$ is in this case the result.* »

« The matrix xx^T may be calculated either by `cbind(x) %*% x` or `x %*% rbind(x)`, since the result of `rbind()` or `cbind()` is always a matrix. However, the best way to compute $x^T x$ or xx^T is `crossprod(x)` or `x %o% x` respectively. »

On veut	Écriture recommandée par le cours
$x^T x$ (un scalaire)	<code>crossprod(x)</code>
xx^T (une matrice)	<code>x %o% x</code>

`x %*% x` **fonctionne** et donne le scalaire — mais par une convention implicite (« *seems implicitly* »), qu'il vaut mieux ne pas invoquer dans du code destiné à durer.

DÉFINITION (§5.7.1).

« The function `crossprod()` forms "cross products", meaning that `crossprod(x, y)` is the same as `t(x) %*% y` but the operation is more efficient. If the second argument to `crossprod()` is omitted it is taken to be the same as the first. »

7.2 Les trois sens de `diag()`

Règle (§5.7.2). « *The meaning of `diag()` depends on its argument.* »

Argument	Ce que rend <code>diag()</code>
un vecteur <code>v</code>	« a diagonal matrix with elements of the vector as the diagonal entries »
une matrice <code>M</code>	« the vector of main diagonal entries of <code>M</code> »
une seule valeur numérique <code>k</code>	« somewhat confusingly ... the k by k identity matrix ! »

« This is the same convention as that used for `diag()` in **Matlab**. »

Le troisième cas est un piège de calcul, pas d'écriture. `diag(3)` rend I_3 , pas la matrice 1×1 contenant 3. Sur une variable — `diag(n)` — la valeur de `n` décide **du sens même** de l'appel : matrice identité si `n` est un scalaire, matrice diagonale si `n` est un vecteur de longueur ≥ 2 . Un code qui marche sur un vecteur de plusieurs éléments **change de comportement** le jour où il n'en reçoit qu'un.

7.3 Résoudre — et le conseil numérique le plus important du chapitre

DÉFINITION (§5.7.2).

« **Solving linear equations is the inverse of matrix multiplication.** When after `b <- A %*% x` only `A` and `b` are given, **the vector `x` is the solution of that linear equation system**. In R, `solve(A, b)` solves the system, returning `x` (up to some accuracy loss). »

« Note that in linear algebra, formally $x = A^{-1}b$... which can be computed by `solve(A)` but rarely is needed. »

« **Numerically, it is both inefficient and potentially unstable to compute** `x <- solve(A) %*% b` **instead of** `solve(A, b)` . » (§5.7.2)

Deux reproches, non un seul : **inefficace** (inverser coûte plus cher que résoudre) et **potentiellement instable** (l'inverse explicite amplifie les erreurs d'arrondi sur une matrice mal conditionnée). La formule mathématique $x = A^{-1}b$ n'est pas la recette numérique.

*« The quadratic form $x^T A^{-1} x$, which is used in multivariate computations, **should be computed by something like** `x %*% solve(A, x)` , **rather than computing the inverse of** `A` . »*

(note 2, §5.7.2) *« **Even better** would be to form a **matrix square root** B with $A = BB^T$ and find **the squared length of the solution of $By = x$** , perhaps using **the Cholesky or eigendecomposition** of A . »* — le cours indique donc **trois niveaux** de qualité numérique, du pire au meilleur : inverse explicite → `solve(A, x)` → racine carrée matricielle.

7.4 Valeurs propres

DÉFINITION (§5.7.3).

*« The function `eigen(Sm)` calculates the eigenvalues and eigenvectors of a **symmetric matrix** S_m . The result of this function is a **list of two components named `values` and `vectors`**. »*

```
ev <- eigen(Sm)
ev$val      # le vecteur des valeurs propres
ev$vec      # la matrice des vecteurs propres correspondants

evals <- eigen(Sm)$values           # si on ne veut que les valeurs
evals <- eigen(Sm, only.values = TRUE)$values # POUR LES GRANDES MATRICES
```

« If the expression `eigen(Sm)` **is used by itself as a command** the two components are printed, with their names. **For large matrices it is better to avoid computing the eigenvectors if they are not needed** by using `only.values = TRUE`. »

Noter que `ev$val` fonctionne par **appariement partiel** de `$` (fiche 302) : la composante s'appelle `values`. Le cours l'utilise sans le dire — c'est commode en interactif, à éviter dans un script.

7.5 Décomposition en valeurs singulières et déterminants

DÉFINITION (§5.7.4).

« The function `svd(M)` takes an arbitrary matrix argument `M` and calculates the singular value decomposition. This consists of a matrix of orthonormal columns `U` with the same column space as `M`, a second matrix of orthonormal columns `V` whose column space is the row space of `M`, and a diagonal matrix of positive entries `D` such that »

$$M = U D V^T \quad \text{soit en R : } M = U \%*\% D \%*\% t(V)$$

« `D` is actually returned as a vector of the diagonal elements. The result of `svd(M)` is a list of three components named `d`, `u` and `v`. »

Le déterminant par la SVD : « If `M` is in fact square, then `absdetM <- prod(svd(M)$d)` calculates the absolute value of the determinant », et « if this calculation were needed often ... it could be defined as an R function » :

```
absdet <- function(M) prod(svd(M)$d)
```

« after which we could use `absdet()` as just another R function. » — l'argument du chapitre 10 (fiche 309), énoncé ici en passant : **une fonction n'est pas d'une autre nature qu'une fonction du système.**

Exercice proposé par le cours (§5.7.4). *« As a further trivial but potentially useful example, you might like to consider writing a function, say `tr()`, to calculate the trace of a square matrix. [Hint : You will not need to use an explicit loop. Look again at the `diag()` function.] »*

Correction pédagogique (le cours ne la donne pas ; elle suit directement de son indication). La trace est la somme des éléments diagonaux ; `diag(M)` rend le vecteur de ces éléments (concept 7.2) ; il ne reste qu'à le sommer :

```
tr <- function(M) sum(diag(M))
```

Et le piège est déjà connu : sur une matrice 1×1 , `diag(M)` reste correct — c'est `diag(k)` sur un scalaire qui bascule vers l'identité. La fonction est donc sûre telle quelle.

*« R has a builtin function `det` to calculate a determinant, **including the sign**, and another, `determinant`, to give **the sign and modulus (optionally on log scale)**. »*

7.6 Moindres carrés et décomposition QR

DÉFINITION (§5.7.5).

« The function `lsfit()` returns a list giving results of **a least squares fitting procedure** ... where `y` is the vector of observations and `x` is the design matrix. See also `ls.diag()` for, among other things, **regression diagnostics**. »

*« Note that **a grand mean term is automatically included and need not be included explicitly as a column of `x`**. Further note that **you almost always will prefer using `lm(.)` to `lsfit()` for regression modelling**. »*

```
Xplus <- qr(X)
b      <- qr.coef(Xplus, y)      # le vecteur de coefficients
fit    <- qr.fitted(Xplus, y)    # la projection orthogonale de y sur l'image de X
res    <- qr.resid(Xplus, y)     # la projection sur le complement orthogonal
```

« `b` is essentially the result of the **Matlab "backslash" operator**. »

« **It is not assumed that `x` has full column rank. Redundancies will be discovered and removed as they are found.** »

« This alternative is **the older, low-level way** to perform least squares calculations. Although still useful in some contexts, **it would now generally be replaced by the statistical models features** » — fiche 313.

● Concept 8 — Assembler : `cbind()` et `rbind()`

DÉFINITION (§5.8).

« Roughly `cbind()` **forms matrices by binding together matrices horizontally, or column-wise**, and `rbind()` **vertically, or row-wise**. »

« The arguments to `cbind()` must be **either vectors of any length, or matrices with the same column size, that is the same number of rows**. The result is a matrix with the concatenated arguments forming the columns. »

« If some of the arguments to `cbind()` are vectors **they may be shorter than the column size** of any matrices present, in which case **they are cyclically extended to match the matrix column size** (or the length of the longest vector if no matrices are given). »

```
X <- cbind(1, X1, X2)  # une colonne de 1, puis X1, puis X2
```

« Suppose `x1` and `x2` have the same number of rows. To combine these by columns into a matrix `x`, **together with an initial column of 1s** » — le `1`, vecteur de longueur 1, est **recyclé** sur toute la hauteur. C'est la colonne d'ordonnée à l'origine d'une matrice de plan.

Règle (§5.8). « The result of `rbind()` or `cbind()` **always has matrix status**. Hence `cbind(x)` and `rbind(x)` **are possibly the simplest ways explicitly to allow the vector `x` to be treated as a column or row matrix** respectively. »

C'est la solution donnée en note 1 du §5.7.1 pour lever l'ambiguïté de `x %**% x`.

● Concept 9 — `c()` contre `cbind()` : ce qui respecte `dim`

Règle (§5.9). « *It should be noted that whereas `cbind()` and `rbind()` are concatenation functions that respect `dim` attributes, the basic `c()` function does not, but rather clears numeric objects of all `dim` and `dimnames` attributes. This is occasionally useful in its own right.* »

Fonction	Effet sur <code>dim</code>
<code>cbind()</code> , <code>rbind()</code>	le respectent — et le résultat est toujours une matrice
<code>c()</code>	efface <code>dim</code> et <code>dimnames</code>

« *The official way to coerce an array back to a simple vector object is to use `as.vector()`* » :

```
vec <- as.vector(X)
vec <- c(X)           # meme resultat, « simply for this side-effect »
```

*« There are **slight differences between the two**, but ultimately **the choice between them is largely a matter of style** (with the former being preferable). »*

● Concept 10 — Tables de fréquences à partir de facteurs

DÉFINITION (§5.10).

« Recall that **a factor defines a partition into groups**. Similarly **a pair of factors defines a two way cross classification**, and so on. The function `table()` **allows frequency tables to be calculated from equal length factors**. If there are k factor arguments, the result is a k -way array of frequencies. »

```
statefr <- table(statef)
```

« gives in `statefr` **a table of frequencies of each state in the sample**. The frequencies are ordered and labelled by the `levels` attribute of the factor. This simple case is equivalent to, but more

convenient than, `statefr <- tapply(statef, statef, length)`. »

EN CLAIR — POURQUOI TABLE() PLUTÔT QUE TAPPLY(..., LENGTH).

Les deux donnent le même vecteur de comptages. `table()` est plus court, et surtout **il rend un objet de classe "table"** — un tableau à une dimension au sens de la fiche 302, avec un `dimnames` nommé. C'est ce qui lui vaut la méthode d'affichage en histogramme observée au concept 5.

La table croisée du cours (§5.10). « Further suppose that `incomef` is a factor giving a suitably defined "**income class**" for each entry in the data vector, for example with the `cut()` function » :

```
factor(cut(incomes, breaks = 35 + 10*(0:7))) -> incomef
table(incomef, statef)
```

incomef \ statef	act	nsw	nt	qld	sa	tas	vic	wa
(35,45]	1	1	0	1	0	0	1	0
(45,55]	1	1	1	1	2	0	1	3
(55,65]	0	3	1	3	2	2	2	1
(65,75]	0	1	0	0	0	0	1	0

« **Extension to higher-way frequency tables is immediate.** »

CONTRÔLE — VÉRIFIER LA TABLE.

Les bornes sont $35 + 10*(0:7)$, soit **35, 45, 55, 65, 75, 85, 95, 105**, d'où des classes semi-ouvertes à gauche : **(35, 45]**, **(45, 55]**, etc. Sur les revenus du chapitre 4 :

- **(35,45]** — 40 (qld), 42 (vic), 41 (nsw), 43 (act) : **quatre** observations, une par état. ligne `1 1 0 1 0 0 1 0`
- **(65,75]** — 69 (vic), 70 (nsw) : **deux** observations. ligne `0 1 0 0 0 0 1 0`
- **Somme totale** : $4 + 10 + 14 + 2 = 30$, l'effectif de l'échantillon.

Et les classes au-delà de 75 sont **absentes du tableau** : aucun revenu ne les atteint. C'est le rôle du `factor()` qui enveloppe `cut()` — il **relit les niveaux effectivement présents** au lieu de conserver sept classes dont trois vides.

Énoncé. Sur le facteur `statef` du chapitre 4, comparer `table(statef)`, `tapply(statef, statef, length)` et `summary(statef)`. Que rend chacun, et lequel choisir ?

Étape 1 — `tapply(statef, statef, length)`. Le cours le donne comme **équivalent** au premier. Le mécanisme est celui de la fiche 304 : regrouper, appliquer `length`, étiqueter. Résultat : **un vecteur nommé** de 8 comptages.

Étape 2 — `table(statef)`. Même contenu numérique. Mais « *the result is a k -way array of frequencies* » : c'est un **tableau à une dimension** (fiche 302), donc muni d'un `dim` de longueur 1 et d'un `dimnames` dont l'élément porte le nom `statef`.

Étape 3 — ce que cette différence change à l'écran. Le `table` s'affiche avec **un en-tête nommé** (le nom de la variable tabulée) ; le résultat de `tapply` n'a que des `names`, qui **ne peuvent pas porter d'en-tête** — c'est exactement la remarque du §3.4.2 : « *the elements of the `dimnames` list may themselves be named, which is not the case for the `names` attribute* ».

Étape 4 — ce que cette différence change au comportement. La classe `"table"` déclenche des **méthodes** : `plot()` produit un histogramme (concept 5), `summary()` un test d'indépendance sur une table croisée. Le vecteur nommé de `tapply` ne déclenche rien.

Étape 5 — la troisième voie. `summary()` sur un facteur rend aussi les effectifs par niveau — mais c'est **une méthode d'affichage**, pas une structure destinée au calcul.

Étape 6 — que choisir. `table()`, sauf raison contraire : il est plus court, il porte l'information de nommage, et il ouvre l'accès aux méthodes. `tapply(..., length)` reste utile quand on veut le

même patron qu'un `tapply(..., mean)` posé juste à côté, pour que les deux résultats soient de même nature (fiche 304).

Comment reconnaître le type d'exercice

Le signal dans l'énoncé	Ce qu'il faut mobiliser
« transformer ce vecteur en matrice »	poser <code>dim</code> , ou <code>matrix()</code> / <code>array()</code>
« ma matrice est remplie dans le mauvais sens »	colonne-majeur → <code>byrow = TRUE</code>
« toute une ligne / colonne »	indice vide : <code>a[2,]</code> , <code>a[, 3]</code>
« le tableau entier »	<code>a[, ,]</code> , <code>a[]</code> , ou <code>a</code> tout seul
« des cases dispersées, sans boucle »	une matrice d'index : une ligne = un jeu de coordonnées
« remplacer une collection irrégulière »	<code>x[i] <- v</code> avec <code>i</code> matrice d'index
« fabriquer des indicatrices »	<code>matrix(0, n, k)</code> + <code>cbind(1:n, facteur)</code> + <code>X[i] <- 1</code>
« compter les couples (bloc, variété) »	<code>table(blocks, varieties)</code> — ou <code>crossprod</code> des indicatrices
« un tableau de zéros »	<code>array(0, dim)</code> — le recyclage à l'extrême
« erreur de longueur en posant <code>dim</code> »	<code>dim<-</code> n'accepte pas le recyclage ; <code>array()</code> oui
« évaluer une fonction sur une grille »	<code>outer(x, y, f)</code>
« toutes les paires de produits »	<code>%o%</code> ou <code>outer(a, b, "*")</code>
« transposer un tableau à 3 indices »	<code>aperm()</code> ; une matrice : <code>t()</code>
« produit matriciel »	<code>%%</code> , jamais <code>*</code>
« $\mathbf{x}^T \mathbf{X}$ »	<code>crossprod(X)</code> — plus efficace que <code>t(X) %% X</code>
« résoudre $\mathbf{Ax} = \mathbf{b}$ »	<code>solve(A, b)</code> , jamais <code>solve(A) %% b</code>
« une forme quadratique $\mathbf{x}^T \mathbf{A}^{-1} \mathbf{x}$ »	<code>x %% solve(A, x)</code>
« valeurs propres seulement, grande matrice »	<code>only.values = TRUE</code>
« le déterminant en valeur absolue »	<code>prod(svd(M)\$d)</code> — ou <code>det()</code> avec le signe
« la trace »	<code>sum(diag(M))</code> — pas de boucle
« ajouter une colonne de 1 »	<code>cbind(1, X)</code> — le 1 est recyclé

Le signal dans l'énoncé	Ce qu'il faut mobiliser
« traiter ce vecteur comme une colonne »	<code>cbind(x)</code> ; comme une ligne : <code>rbind(x)</code>
« repasser en vecteur simple »	<code>as.vector(X)</code>
« une table de fréquences croisée »	<code>table(f1, f2)</code>
« découper une variable continue en classes »	<code>cut()</code> , enveloppé dans <code>factor()</code>

Comment résoudre ce type d'exercice

Protocole « construire un tableau » — 4 étapes.

1. Choisir **la voie** : `array(donnees, dim)` **recycle** ; `dim(x) <- d` **exige la bonne longueur**.
2. Vérifier que **le produit des dimensions égale la longueur** — R le vérifie pour vous, autant le savoir avant.
3. Se rappeler l'ordre : **premier indice le plus rapide**. Pour remplir ligne par ligne, `matrix(..., byrow = TRUE)`.
4. Contrôler avec `dim()` et un affichage sur un petit cas avant de passer à l'échelle.

Protocole « extraire ou modifier une collection irrégulière » — 5 étapes.

1. Écrire les coordonnées voulues **une par ligne**.
2. En faire une **matrice numérique** à autant de colonnes que le tableau a de dimensions — `cbind()` est le moyen le plus direct.
3. Vérifier : **pas d'indice négatif** ; un zéro **ignore** la ligne ; un `NA` **produit** un `NA`.
4. `x[i]` pour extraire, `x[i] <- v` pour remplacer.
5. Vérifier que la matrice est bien **numérique** — une matrice logique ou de caractères serait traitée comme **un simple vecteur d'index**.

Protocole « calcul matriciel numériquement sain » — 4 étapes.

1. **Ne jamais inverser pour résoudre** : `solve(A, b)` , pas `solve(A) %*% b` — « *both inefficient and potentially unstable* ».
2. Préférer `crossprod()` à `t(x) %*% x`.
3. Pour une forme quadratique avec inverse : `x %*% solve(A, x)` ; mieux encore, passer par une **racine carrée matricielle** (Cholesky, décomposition spectrale).
4. Ne calculer **que ce dont on a besoin** : `eigen(Sm, only.values = TRUE)` sur les grandes matrices.

● Common mistakes

Erreur	Correction
Croire qu'un tableau est un type à part	c'est un vecteur + <code>dim</code>
Remplir une matrice « ligne par ligne »	l'ordre est colonne-majeur → <code>byrow = TRUE</code>
Utiliser <code>dim<-</code> avec un vecteur trop court	erreur de longueur → <code>array()</code> , qui recycle
Oublier qu' <code>a[2,,]</code> perd une dimension	c'est <code>drop</code> (fiche 302) → <code>drop = FALSE</code>
Croire <code>x[i]</code> toujours « à plat »	pas si <code>i</code> est une matrice
Mettre des indices négatifs dans une matrice d'index	interdit
Croire qu'une ligne à zéro sélectionne	elle est ignorée ; une ligne à <code>NA</code> produit <code>NA</code>
Passer une matrice logique comme matrice d'index	elle est traitée comme un vecteur d'index
Ajouter un vecteur plus long qu'une matrice	erreur (règle 4) ; plus court → recyclé en silence
Utiliser <code>*</code> pour un produit matriciel	c'est <code>%%</code> ; <code>*</code> est élément par élément
Écrire <code>t(X) %% y</code>	<code>crossprod(X, y)</code> est plus efficace
Compter sur <code>x %% x</code>	ambigu → <code>crossprod(x)</code> ou <code>x %o% x</code>
Appeler <code>diag(n)</code> sur un scalaire en attendant une matrice 1×1	c'est la matrice identité $n \times n$
Écrire <code>solve(A) %% b</code>	« <i>inefficient and potentially unstable</i> » → <code>solve(A, b)</code>
Calculer A^{-1} pour une forme quadratique	<code>x %% solve(A, x)</code>
Calculer les vecteurs propres sans en avoir besoin	<code>only.values = TRUE</code>
Croire que <code>svd()\$d</code> est une matrice	« <i>actually returned as a vector of the diagonal elements</i> »
Croire <code>prod(svd(M)\$d)</code> égal au déterminant	c'est sa valeur absolue → <code>det()</code> pour le signe
Écrire une boucle pour la trace	<code>sum(diag(M))</code>

Erreur	Correction
Ajouter une colonne de 1 à la main	<code>cbind(1, X)</code> suffit — recyclage
Utiliser <code>c()</code> pour assembler des matrices	il efface <code>dim</code> → <code>cbind()</code> / <code>rbind()</code>
Oublier que <code>cbind(x)</code> rend une matrice	c'est précisément son usage
Inclure une colonne de 1 dans <code>lsfit()</code>	« a grand mean term is automatically included »
Utiliser <code>lsfit()</code> pour modéliser	« you almost always will prefer using <code>Lm(.)</code> »
Garder des classes vides après <code>cut()</code>	envelopper dans <code>factor()</code>

Ultimate Review

Le tableau. « A **dimension vector** is a vector of non-negative integers. **If its length is k then the array is k -dimensional.** » Un vecteur devient tableau **par son attribut** `dim` ; `dim(Z)` se lit **et s'écrit**. Une matrice est un tableau à deux indices.

L'ordre. Colonne-majeur, comme en FORTRAN : « the **first subscript moving fastest** and the **last subscript slowest** ». Pour `dim = c(3,4,2)` : `a[1,1,1]` , `a[2,1,1]` , ..., `a[3,4,2]` . Longueur = **produit des extensions** ; une extension **peut être nulle**.

Indexer. Indices séparés par des **virgules** · **indice vide = toute l'étendue** · `a[, , ]` = `a` · **un seul indice** → `dim` **ignoré**, « this is **not the case** ... if the single index is **itself an array** ».

Matrice d'index. **Autant de colonnes que de dimensions**, autant de lignes qu'on veut · **une ligne = un jeu de coordonnées** · sert à **extraire** (`x[i]`) et à **remplacer** (`x[i] <- v`) une collection irrégulière · **négatifs interdits, ligne à zéro ignorée, ligne à NA** → `NA`, et **doit être numérique** — sinon elle est traitée comme un simple vecteur d'index.

Construire. `array(v, dim)` **recycle** si `v` est trop court ; `dim(v) <- d` **lève une erreur**. `array(0, c(3,4,2))` = « an extreme but common example ». `Z[]` et `Z` seul = le tableau entier.

Recyclage mixte — les cinq règles (§5.4.1). (1) lecture **de gauche à droite** · (2) les **vecteurs courts** sont **recyclés** · (3) tant qu'on n'a que vecteurs courts et tableaux, **tous les tableaux doivent avoir le même `dim`** · (4) **un vecteur plus long qu'un tableau est une erreur** · (5) le résultat est **un tableau au `dim` commun**.

Produit extérieur. `a %o% b` = `outer(a, b, "*")` · le `dim` du résultat est la **concaténation des deux `dim`** (« order is important ») · **la fonction est remplaçable** : `outer(x, y, f)` évalue `f` sur **toutes les paires** — le remplacement idiomatique de la double boucle · **non commutatif**.

Transposer. `t(A)` pour une matrice · `aperm(a, perm)` en dimension quelconque, « *a generalization of transposition* », avec `perm` **permutation de $\{1, \dots, k\}$** .

Algèbre. `nrow` `ncol` · `%%` produit matriciel, `*` élément par élément · `crossprod(X, y) = t(X) %*% y` **en plus efficace**, second argument omis = le premier · `x %*% x` **est ambigu** → `crossprod(x)` pour $x^T x$, `x %o% x` pour xx^T · `diag()` **a trois sens** : vecteur → **matrice diagonale**, matrice → **vecteur diagonal**, scalaire k → **identité $k \times k$** .

Résoudre. `solve(A, b)` — « *it is both inefficient and potentially unstable to compute `solve(A) %*% b`* » · forme quadratique : `x %*% solve(A, x)` ; mieux : **racine carrée matricielle** (Cholesky ou décomposition spectrale).

Décompositions. `eigen(Sm)` sur une matrice **symétrique** → liste `values` / `vectors` ; `only.values = TRUE` sur les grandes · `svd(M)` sur une matrice **quelconque** → liste `d`, `u`, `v`, avec $M = UDV^T$ et `d` **rendu comme vecteur** ; `prod(svd(M)$d)` = **valeur absolue** du déterminant · `det()` donne le signe, `determinant()` signe et module (**au besoin en échelle log**) · `qr()` + `qr.coef` / `qr.fitted` / `qr.resid` — « *it is not assumed that `x` has full column rank* » · `lsfit()` ajoute **automatiquement** la constante ; « *you almost always will prefer `Lm(.)`* ».

Assembler. `cbind()` horizontal, `rbind()` vertical · arguments : **vecteurs de n'importe quelle longueur, ou matrices de même nombre de lignes** · les vecteurs courts sont **étendus cycliquement** · **le résultat est toujours une matrice**, d'où `cbind(x) / rbind(x)` pour forcer colonne ou ligne.

`c()` **contre** `cbind()` · « *`cbind()` and `rbind()` respect `dim` attributes ... `c()` ... clears numeric objects of all `dim` and `dimnames`* » · la voie officielle pour aplatir est `as.vector(X)`, « *with the former being preferable* » devant `c(X)`.

Tables. `table(f)` — fréquences **ordonnées et étiquetées par** `levels` · `table(f1, f2)` = classification croisée ; k **facteurs** → **tableau à k dimensions** · équivalent mais plus commode que `tapply(f, f, length)` · `cut()` découpe une variable continue, à envelopper dans `factor()` · `plot()` sur une "table" produit **un histogramme** — le dispatch S3.

Les chiffres du chapitre. `dim = c(3,4,2)` → **24** entrées · `x <- array(1:20, c(4,5))`, `x[i]` = **9, 6, 3** · déterminants : **100** produits, **10 000** déterminants, « *about 1 in 20 ... is singular* » · table croisée : **4 + 10 + 14 + 2 = 30** observations.

Active Recall

« *A dimension vector is a vector of non-negative integers. If its length is k then the array is k -dimensional, e.g. a matrix is a 2-dimensional array. The dimensions are indexed*

from one up to the values given in the dimension vector. » (§5.1)

« **A vector can be used by R as an array only if it has a dimension vector as its `dim` attribute.** » Sur un vecteur `z` de 1500 éléments, `dim(z) <- c(3,5,100)` « gives it the `dim` attribute that **allows it to be treated as a 3 by 5 by 100 array** ».

Aucune donnée n'est déplacée : c'est un attribut, pas une conversion (fiche 303).

« *The values in the data vector give the values in the array **in the same order as they would occur in FORTRAN**, that is "**column major order**," with the first subscript moving fastest and the last subscript slowest.* » (§5.1)

Pour `dim = c(3,4,2)`, les $3 \times 4 \times 2 = 24$ entrées sont rangées dans l'ordre

`a[1,1,1], a[2,1,1], ..., a[2,4,2], a[3,4,2]`

Le premier indice tourne le plus vite : on descend la colonne avant de passer à la suivante. C'est l'inverse de la lecture naturelle d'un tableau imprimé — d'où l'argument `byrow = TRUE` de `matrix()`.

« `a[2,,]` is **a 4 by 2 array** with dimension vector `c(4,2)` and data vector containing the values

```
c(a[2,1,1], a[2,2,1], a[2,3,1], a[2,4,1],  
  a[2,1,2], a[2,2,2], a[2,3,2], a[2,4,2])
```

in that order. » (§5.2)

Deux choses à voir. (1) **L'indice vide prend toute l'étendue** : « *if any index position is given an empty index vector, then **the full range of that subscript is taken*** ». (2) **La dimension de longueur 1 disparaît** — le résultat est 4×2 , pas $1 \times 4 \times 2$: c'est la règle `drop` (fiche 302), appliquée sans qu'on l'ait demandée.

Et l'ordre suit toujours colonne-majeur, la 2^e dimension d'origine tournant maintenant le plus vite.

« if an array name is given with **just one subscript or index vector**, then **the corresponding values of the data vector only are used ; in this case the dimension vector is ignored**. **This is not the case, however, if the single index is not a vector but itself an array.** » (§5.2)

- **i vecteur** → on lit le **vecteur sous-jacent**, `dim` mis de côté (le `m[1]` de la fiche 302).
- **i matrice** → **matrice d'index** : « *two columns and as many rows as desired* », chaque ligne étant **un jeu de coordonnées**.

Et « ***Index matrices must be numerical : any other form of matrix (e.g. a logical or character matrix) supplied as a matrix is treated as an indexing vector*** » (§5.3).
Une matrice logique **retombe** dans le premier cas.

9 6 3, comme le montre le cours.

Vérification. Le remplissage est colonne-majeur, donc l'élément (l, j) d'une matrice 4×5 remplie par `1:20` vaut $4(j - 1) + l$:

Coordonnées	Calcul	Valeur
(1, 3)	$4 \times 2 + 1$	9
(2, 2)	$4 \times 1 + 2$	6
(3, 1)	$4 \times 0 + 3$	3

Et `x[i] <- 0` remplace **exactement ces trois cases**, ce que confirme l'affichage du cours.

« if `h` is **shorter than 24**, its values are recycled from the beginning again to make it up to size 24 ... but `dim(h) <- c(3,4,2)` would signal an error about mismatching length. » (§5.4)

	vecteur trop court
<code>array(v, dim)</code>	recycle
<code>dim(v) <- d</code>	erreur

« If the size of `h` is **exactly 24** the result is the same. »

Le cas extrême et courant : `array(0, c(3,4,2))` — un seul zéro recyclé 24 fois, pour obtenir un tableau nul.

« The precise rule ... is **somewhat quirky and hard to find in the references**. From experience we have found the following to be a reliable guide » (§5.4.1) :

1. « The expression is **scanned from left to right**. »
2. « Any **short vector** operands are **extended by recycling** their values until they match the size of any other operands. »
3. « As long as short vectors and arrays only are encountered, **the arrays must all have the same `dim` attribute or an error results**. »
4. « **Any vector operand longer than a matrix or array operand generates an error**. »
5. « If array structures are present ... **the result is an array structure with the common `dim` attribute** of its array operands. »

Les règles 2 et 4 sont asymétriques : trop court → recyclé **en silence** ; trop long → **erreur**. C'est le premier cas qui blesse.

*« their outer product is an array whose **dimension vector is obtained by concatenating their two dimension vectors (order is important)**, and whose data vector is got by **forming all possible products of elements** of the data vector of `a` with those of `b` »* (§5.5).

```
ab <- a %o% b           # ou outer(a, b, "**")
```

« *The multiplication function can be replaced by an arbitrary function of two variables.*

» D'où :

```
f <- function(x, y) cos(y)/(1 + x^2)
z <- outer(x, y, f)
```

`outer` évalue `f` sur **toutes les paires** (x_i, y_j) — le remplacement exact d'une double boucle, mais **vectorisé**. Le cours le dit du problème des déterminants : « *The "obvious" way ... with `for` loops ... is **so inefficient as to be impractical**.* »

« *the outer product operator is **of course non-commutative*** ».

```
d <- outer(0:9, 0:9)          # 100 produits a*d
fr <- table(outer(d, d, "-"))  # 10 000 différences ad - bc
plot(fr, xlab = "Determinant", ylab = "Frequency")
```

Le premier `outer` utilise la multiplication par défaut : `d` contient les 100 produits possibles. **Le même objet sert pour *ad* et pour *bc***, puisque les deux parcourent le même ensemble avec les mêmes multiplicités.

Le second `outer` forme les $100 \times 100 = 10\,000$ différences — exactement le nombre de quadruplets (a, b, c, d) dans $\{0, \dots, 9\}^4$.

« Notice that `plot()` **here uses a histogram like plot method, because it "sees" that `fr` is of class `"table"`** » — le dispatch S3.

« *It is also perhaps surprising that **about 1 in 20 such matrices is singular**.* » (Enrichissement : le compte exact est 570 sur 10 000, soit 5,7 %, dont **361 dus au seul produit nul** — le zéro est la valeur spéciale.)

« *The meaning of `diag()` depends on its argument.* » (§5.7.2)

Argument	Résultat
un vecteur <code>v</code>	une matrice diagonale dont les entrées diagonales sont celles de <code>v</code>
une matrice <code>M</code>	le vecteur des entrées de la diagonale principale
un scalaire <code>k</code>	« somewhat confusingly ... the k by k identity matrix ! »

« This is the same convention as that used for `diag()` in **Matlab**. »

Le danger est dans le troisième cas : `diag(n)` change de **sens** selon que `n` est un scalaire ou un vecteur. Du code correct sur un vecteur de plusieurs éléments bascule le jour où il n'en reçoit qu'un.

C'est aussi le deuxième sens qui donne la trace : `sum(diag(M))` .

« Note that in linear algebra, formally $x = A^{-1}b$... which can be computed by `solve(A)` **but rarely is needed. Numerically, it is both inefficient and potentially unstable to compute** `x <- solve(A) %*% b` **instead of** `solve(A, b)` . » (§5.7.2)

Deux reproches distincts : *inefficace* (inverser coûte plus cher que résoudre) et *potentiellement instable* (l'inverse explicite amplifie les erreurs d'arrondi sur une matrice mal conditionnée). **La formule mathématique n'est pas la recette numérique.**

Pour une forme quadratique : « $x^T A^{-1} x$... **should be computed by something like** `x %*% solve(A, x)` ». Et mieux encore (note 2) : former **une racine carrée matricielle** B avec $A = BB^T$ et prendre la longueur au carré de la solution de $By = x$, « *perhaps using the Cholesky or eigendecomposition* ».

Note 1 du §5.7.1 : « `x %*% x` **is ambiguous**, as it could mean either $x^T x$ or xx^T , where x is the column form. **In such cases the smaller matrix seems implicitly to be the interpretation adopted, so the scalar $x^T x$ is in this case the result.** »

On veut	Le cours recommande
$x^T x$ (scalaire)	<code>crossprod(x)</code>
xx^T (matrice)	<code>x %o% x</code>

Et pour forcer explicitement la forme : `cbind(x) %*% x` ou `x %*% rbind(x)` , « *since **the result of `rbind()` or `cbind()` is always a matrix*** ».

Le mot « *seems* » est ce qui doit alerter : c'est une convention observée, pas une garantie à invoquer dans du code durable.

`eigen(Sm)` — sur une matrice **symétrique** — rend « *a list of two components named `values` and `vectors`* ». Pour les grandes matrices : « *it is better to avoid computing the eigenvectors if they are not needed* » → `eigen(Sm, only.values = TRUE)$values` .

`svd(M)` — sur « *an arbitrary matrix argument* » — rend « *a list of three components named `d`, `u` and `v`* », avec $M = UDV^T$ et « *D is actually returned as a vector of the diagonal elements* ».

Et : `prod(svd(M)$d)` donne **la valeur absolue** du déterminant. Pour le signe, R a `det()` ; pour signe et module (au besoin **en échelle log**), `determinant()` .

« *whereas `cbind()` and `rbind()` are concatenation functions that respect `dim` attributes, the basic `c()` function does not, but rather clears numeric objects of all `dim` and `dimnames` attributes. This is occasionally useful in its own right.* » (§5.9)

C'est cohérent avec la table de la fiche 303 : la coercion **efface tout**.

« *The official way to coerce an array back to a simple vector object is to use `as.vector()`* » ; `c(X)` donne un résultat similaire « *simply for this side-effect* », mais « *there are slight differences ... with the former being preferable* ».

Et la contrepartie utile : « *The result of `rbind()` or `cbind()` always has matrix status* » — d'où `cbind(x)` pour forcer un vecteur en colonne.

« *The function `table()` allows frequency tables to be calculated from equal length factors. If there are k factor arguments, the result is a k -way array of frequencies.* » (§5.10) « *The frequencies are ordered and labelled by the `Levels` attribute of the factor.* »

Pour découper une variable continue en classes :

```
factor(cut(incomes, breaks = 35 + 10*(0:7))) -> incomef  
table(incomef, statef)
```

Les bornes sont **35, 45, ..., 105**, d'où des classes **(35, 45]**, **(45, 55]**... Le `factor()` **enveloppant** relit les niveaux effectivement présents et **écarte les classes vides**.

Le cas à un facteur est « *equivalent to, but more convenient than* `tapply(statef, statef, length)` » — mais **le résultat n'est pas du même type** : `table()` rend un objet de classe `"table"`, donc un tableau à une dimension, qui déclenche les méthodes (l'histogramme du concept 5).

Flashcards

Question	Réponse
Qu'est-ce qu'un tableau ?	Une collection de données à indices multiples
Comment un vecteur devient-il un tableau ?	Par son attribut <code>dim</code>
Qu'est-ce qu'un vecteur de dimension ?	Un vecteur d'entiers non négatifs
Sa longueur k signifie ?	Un tableau à k dimensions
Une matrice, c'est ?	Un tableau à 2 dimensions
L'ordre de remplissage ?	Colonne-majeur , comme en FORTRAN
Quel indice varie le plus vite ?	Le premier
Combien d'entrées pour <code>dim = c(3,4,2)</code> ?	24
Comment remplir ligne par ligne ?	<code>byrow = TRUE</code>
Que vaut la longueur du vecteur sous-jacent ?	Le produit des extensions
Une extension peut-elle être nulle ?	Oui
Que fait un indice vide ?	Il prend toute l'étendue
Que vaut <code>a[, ,]</code> ?	Le tableau entier , comme <code>a</code> seul
Lire et écrire les dimensions ?	<code>dim(Z)</code> , des deux côtés d'une assignation
Que fait <code>x[i]</code> avec <code>i</code> vecteur ?	<code>dim</code> est ignoré — le vecteur sous-jacent
Et avec <code>i</code> matrice ?	Une matrice d'index
Combien de colonnes doit-elle avoir ?	Autant que de dimensions
Que représente une ligne ?	Un jeu de coordonnées
Longueur du résultat ?	Le nombre de lignes de l'index
Indices négatifs dans une matrice d'index ?	Interdits
Ligne contenant un zéro ?	Ignorée

Question	Réponse
Ligne contenant un <code>NA</code> ?	Produit un <code>NA</code>
Type obligatoire d'une matrice d'index ?	Numérique
Une matrice logique passée en index ?	Traitée comme un vecteur d'index
À quoi sert <code>x[i] <- v</code> ?	Remplacer une collection irrégulière
Comment fabriquer une matrice d'indicateurs ?	<code>matrix(0, n, k) + cbind(1:n, f) + X[i] <- 1</code>
La matrice d'incidence par les indicateurs ?	<code>crossprod(Xb, Xv)</code>
La voie directe ?	<code>table(blocks, varieties)</code>
<code>array(v, dim)</code> si <code>v</code> est trop court ?	Il recycle
<code>dim(v) <- d</code> dans le même cas ?	Erreur de longueur
Un tableau de zéros ?	<code>array(0, dim)</code>
Règle 1 du calcul mixte ?	Lecture de gauche à droite
Règle 2 ?	Les vecteurs courts sont recyclés
Règle 3 ?	Tous les tableaux de même dim , sinon erreur
Règle 4 ?	Un vecteur plus long qu'un tableau → erreur
Règle 5 ?	Le résultat est un tableau au dim commun
L'opérateur de produit extérieur ?	<code>%o%</code>
Son équivalent fonctionnel ?	<code>outer(a, b, "**")</code>
Le <code>dim</code> du résultat ?	La concaténation des deux <code>dim</code>
Peut-on remplacer la multiplication ?	Oui , par toute fonction de deux variables
Évaluer f sur une grille ?	<code>outer(x, y, f)</code>
<code>%o%</code> est-il commutatif ?	Non
Transposer une matrice ?	<code>t(A)</code>

Question	Réponse
Permuter les dimensions d'un tableau ?	<code>aperm(a, perm)</code>
Que doit être <code>perm</code> ?	Une permutation de $\{1, \dots, k\}$
Nombre de lignes, de colonnes ?	<code>nrow()</code> , <code>ncol()</code>
Produit matriciel ?	<code>%%</code>
Produit élément par élément ?	<code>*</code>
Une forme quadratique ?	<code>x %%% A %%% x</code>
Que vaut <code>crossprod(X, y)</code> ?	<code>t(X) %%% y</code> , en plus efficace
Et si le 2 ^e argument est omis ?	Il vaut le premier
Pourquoi <code>x %%% x</code> est-il ambigu ?	$\mathbf{x}^T \mathbf{x}$ ou $\mathbf{x} \mathbf{x}^T$
Quelle interprétation R adopte-t-il ?	La plus petite matrice — le scalaire
Comment obtenir $\mathbf{x}^T \mathbf{x}$ proprement ?	<code>crossprod(x)</code>
Et $\mathbf{x} \mathbf{x}^T$?	<code>x %o% x</code>
<code>diag(v)</code> avec <code>v</code> vecteur ?	Une matrice diagonale
<code>diag(M)</code> avec <code>M</code> matrice ?	Le vecteur diagonal
<code>diag(k)</code> avec <code>k</code> scalaire ?	L'identité $k \times k$
Quelle convention suit <code>diag()</code> ?	Celle de Matlab
Comment calculer la trace ?	<code>sum(diag(M))</code>
Résoudre $\mathbf{Ax} = \mathbf{b}$?	<code>solve(A, b)</code>
Pourquoi pas <code>solve(A) %%% b</code> ?	« <i>inefficient and potentially unstable</i> »
Une forme quadratique avec inverse ?	<code>x %%% solve(A, x)</code>
Mieux encore ?	Une racine carrée matricielle (Cholesky)
Que rend <code>eigen()</code> ?	Une liste <code>values</code> / <code>vectors</code>

Question	Réponse
Sur quel type de matrice ?	Symétrique
L'économie sur les grandes matrices ?	<code>only.values = TRUE</code>
Que rend <code>svd()</code> ?	Une liste <code>d</code> , <code>u</code> , <code>v</code>
La relation ?	$M = UDV^T$
Sous quelle forme est <code>d</code> ?	Un vecteur , pas une matrice
Que vaut <code>prod(svd(M)\$d)</code> ?	La valeur absolue du déterminant
Le déterminant avec son signe ?	<code>det()</code>
Signe et module, en log ?	<code>determinant()</code>
Les trois fonctions QR ?	<code>qr.coef</code> · <code>qr.fitted</code> · <code>qr.resid</code>
<code>x</code> doit-il être de rang plein ?	Non — les redondances sont retirées
Que fait <code>lsfit()</code> automatiquement ?	Il inclut la constante
Que préférer pour modéliser ?	<code>lm()</code>
<code>cbind()</code> assemble comment ?	Horizontalement , par colonnes
<code>rbind()</code> ?	Verticalement , par lignes
Contrainte sur les matrices passées à <code>cbind()</code> ?	Même nombre de lignes
Un vecteur trop court ?	Il est étendu cycliquement
Comment ajouter une colonne de 1 ?	<code>cbind(1, X1, X2)</code>
Quel est le statut du résultat ?	Toujours une matrice
Forcer un vecteur en colonne ?	<code>cbind(x)</code>
En ligne ?	<code>rbind(x)</code>
<code>c()</code> respecte-t-il <code>dim</code> ?	Non — il l' efface
La voie officielle pour aplatir ?	<code>as.vector(X)</code>

Question	Réponse
Et <code>c(X)</code> ?	Même effet, mais <code>as.vector</code> est préférable
Que fait <code>table(f)</code> ?	Une table de fréquences
Comment sont ordonnées les fréquences ?	Par l'attribut <code>levels</code>
<code>table(f1, f2)</code> ?	Une classification croisée
k facteurs ?	Un tableau à k dimensions
L'équivalent avec <code>tapply</code> ?	<code>tapply(f, f, length)</code> — moins commode
Découper une variable continue ?	<code>cut()</code>
Pourquoi l'envelopper dans <code>factor()</code> ?	Pour écarter les classes vides
Que fait <code>plot()</code> sur un objet <code>"table"</code> ?	Un histogramme — dispatch S3